The most important thing for you to be aware of, as a developer of a UWP DirectX app, is that you must enable your app thread to dispatch MTA threads by setting **Platform::MTAThread** on **main()**.

```
C++
```

```
[Platform::MTAThread]
int main(Platform::Array<Platform::String^>^)
{
    auto myDXAppSource = ref new MyDXAppSource(); // your view provider
factory
    CoreApplication::Run(myDXAppSource);
    return 0;
}
```

When the app object for your UWP DirectX app activates, it creates the ASTA that will be used for the UI view. The new ASTA thread calls into your view provider factory, to create the view provider for your app object, and as a result, your view provider code will run on that ASTA thread.

Also, any thread that you spin off from the ASTA must be in an MTA. Be aware that any MTA threads that you spin off can still create reentrancy issues and result in a deadlock.

If you're porting existing code to run on the ASTA thread, keep these considerations in mind:

- Wait primitives, such as **CoWaitForMultipleObjects**, behave differently in an ASTA than in an STA.
- The COM call modal loop operates differently in an ASTA. You can no longer receive unrelated calls while an outgoing call is in progress. For example, the following behavior will create a deadlock from an ASTA (and immediately crash the app):

    1. The ASTA calls an MTA object and passes an interface pointer P1.
    2. Later, the ASTA calls the same MTA object. The MTA object calls P1 before it returns to the ASTA.
    3. P1 cannot enter the ASTA as it's blocked making an unrelated call. However, the MTA thread is blocked as it tries to make the call to P1.

    You can resolve this by :
    - Using the **async** pattern defined in the Parallel Patterns Library (PPLTasks.h)
    - Calling **CoreDispatcher::ProcessEvents** from your app's ASTA (the main thread of your app) as soon as possible to allow arbitrary calls.

> That said, you cannot rely on immediate delivery of unrelated calls to your app's ASTA. For more info about async calls, read Asynchronous programming in C++.

Overall, when designing your UWP app, use the **CoreDispatcher** for your app's **CoreWindow** and **CoreDispatcher::ProcessEvents** to handle all UI threads rather than trying to create and manage your MTA threads yourself. When you need a separate thread that you cannot handle with the **CoreDispatcher**, use async patterns and follow the guidance mentioned earlier to avoid reentrancy issues.

Project templates and tools for games

Article • 10/20/2022

This topic discusses what you need to start programming DirectX games for the Universal Windows Platform (UWP).

Visual Studio

Download and install Microsoft Visual Studio.

For info about setting up Visual Studio for C++/WinRT development, see Visual Studio support for C++/WinRT.

Topics in this section

Topic	Description
DirectX game project templates	Learn about the templates for creating a UWP and DirectX game.
Visual Studio tools for game programming	An overview of DirectX specific tools available in Visual Studio.
Graphics diagnostics tools	Learn how to get and use the graphics diagnostics features including Graphics Debugging, Graphics Frame Analysis, and GPU Usage in Visual Studio.

Next steps

If you're porting an existing game, see these topics.

- Port from OpenGL ES 2.0 to DirectX 11
- Port from DirectX 9 to UWP

If you're creating a new DirectX game, see these topics.

- Create a simple UWP game with DirectX
- Developing Marble Maze, a Universal Windows Platform game in C++ and DirectX

DirectX game project templates

Article • 10/20/2022

The DirectX and Universal Windows Platform (UWP) templates allow you to quickly create a project as a starting point for your game.

Prerequisites

To create the project you need to:

- Download Microsoft Visual Studio 2015. Visual Studio 2015 has tools for graphics programming, such as debugging tools. For an overview of DirectX graphics and gaming features and tools, see Visual Studio tools for DirectX game development.

Choosing a template

Visual Studio 2015 includes three DirectX and UWP templates:

- DirectX 11 App (Universal Windows) - The DirectX 11 App (Universal Windows) template creates a UWP project, which renders directly to an app window using DirectX 11.
- DirectX 12 App (Universal Windows) - The DirectX 12 App (Universal Windows) template creates a project UWP, which renders directly to an app window using DirectX 12.
- DirectX 11 and XAML App (Universal Windows) - The DirectX 11 and XAML App (Universal Windows) template creates a UWP project, which renders inside a XAML control using DirectX 11. This template uses a **SwapChainPanel**, so you can use XAML UI controls. This can make adding user interface elements easier, but using the XAML template may result in lower performance.

Which template you choose depends on the performance and what technologies you want to use.

Template structure

The DirectX Universal Windows templates contain the following files:

- pch.h and pch.cpp - Precompiled header support.
- Package.appxmanifest - The properties of the deployment package for the app.

- *.pfx - Certificates for the application.
- External Dependencies - Links to external files the project use.s
- *Main.h and *Main.cpp - Methods for managing application assets, updating application state, and rendering the frame.
- App.h and App.cpp - Main entry point for the application. Connects the app with the Windows shell and handles application lifecycle events. These files only appear in the DirectX 11 App (Universal Windows) and DirectX 12 App (Universal Windows) templates.
- App.xaml, App.xaml.cpp, and App.xaml.h - Main entry point for the application. Connects the app with the Windows shell and handles application lifecycle events. These files only appear in the DirectX 11 and XAML App (Universal Windows) template.
- DirectXPage.xaml, DirectXPage.xaml.cpp, and DirectXPage.xaml.h - A page that hosts a DirectX SwapChainPanel. These files only appear in the DirectX 11 and XAML App (Universal Windows) template.
- Content
  - Sample3DSceneRenderer.h and Sample3DSceneRenderer.cpp - A sample renderer that instantiates a basic rendering pipeline.
  - SampleFpsTextRenderer.h and SampleFpsTextRenderer.cpp - Renders the current FPS value in the bottom right corner of the screen using Direct2D and DirectWrite. These files only appear in the DirectX 11 App (Universal Windows) and DirectX 11 and XAML App (Universal Windows) templates.
  - SamplePixelShader.hlsl - A simple example pixel shader.
  - SampleVertexShader.hlsl - A simple example vertex shader.
  - ShaderStructures.h - Structures used to send date to the example vertex shader.
- Common
  - StepTimer.h - A helper class for animation and simulation timing.
  - DirectXHelper.h - Misc Helper functions.
  - DeviceResources.h and Device Resources.cpp - Provides an interface for an application that owns DeviceResources to be notified of the device being lost or created.
  - d3dx12.h - Contains the D3DX12 utility library. This file only appears in the DirectX 12 App (Universal Windows).
- Assets - Logo and splashscreen images used by the application.

Next steps

Now that you have a starting point, add to it to build your game development knowledge and Microsoft Store game development skills.

If you are porting an existing game, see the following topics.

- Port from OpenGL ES 2.0 to Direct3D 11.1
- Port from DirectX 9 to Universal Windows Platform

If you are creating a new DirectX game, see the following topics.

- Create a simple UWP game with DirectX
- Developing Marble Maze, a Universal Windows Platform game in C++ and DirectX

Visual Studio tools for game programming

Article • 10/20/2022

Summary

- Create a DirectX game project from a template
- Visual Studio tools for DirectX game programming

If you use Visual Studio Ultimate to develop DirectX apps, there are additional tools available for creating, editing, previewing, and exporting image, model, and shader resources. There are also tools that you can use to convert resources at build time and debug DirectX graphics code.

This topic gives an overview of these graphics tools.

Image Editor

Use the Image Editor to work with the kinds of rich texture and image formats that DirectX uses. The Image Editor supports the following formats.

- .png
- .jpg, .jpeg, .jpe, .jfif
- .dds
- .gif
- .bmp
- .dib
- .tif, .tiff
- .tga

Create build customization files to convert these to .dds files at build time.

For more information, see Working with Textures and Images.

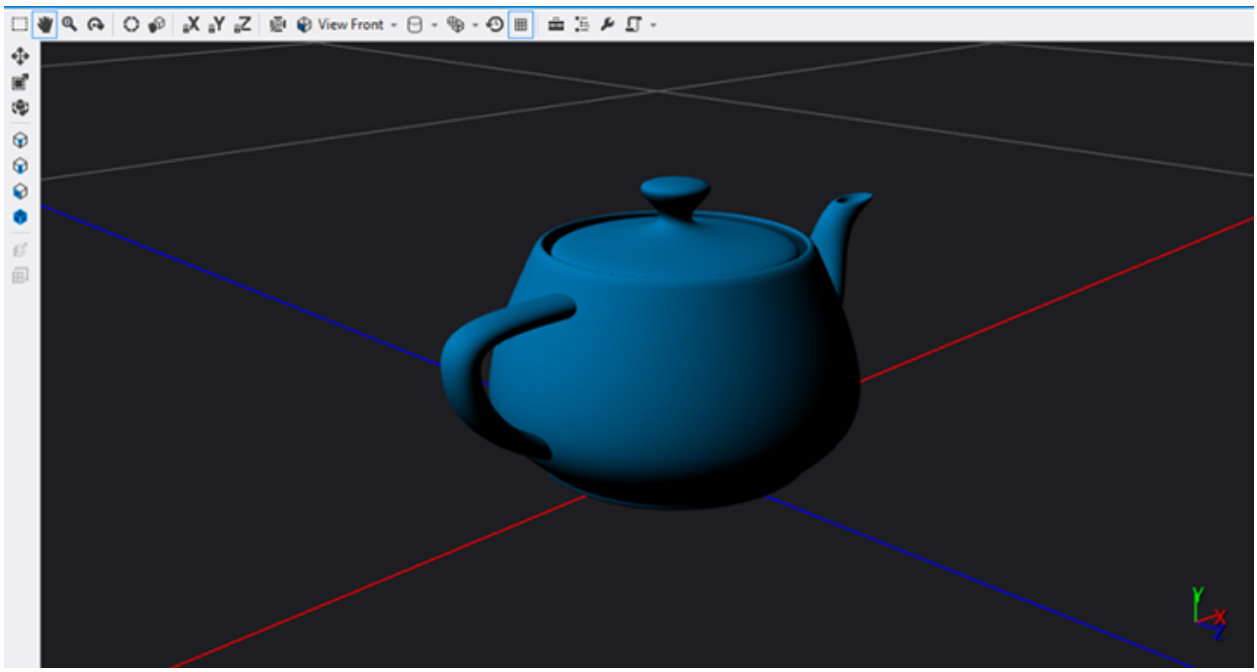
> **Note** The Image Editor is not intended to be a replacement for a full feature image editing app, but is appropriate for many simple viewing and editing scenarios.

Model Editor

You can use the Model Editor to create basic 3D models from scratch, or to view and modify more-complex 3D models from full-featured 3D modeling tools. The Model Editor supports several 3D model formats that are used in DirectX app development. You can create build customization files to convert these to .cmo files at build time.

- .fbx
- .dae
- .obj

Here's a screenshot of a model in the editor with lighting applied.



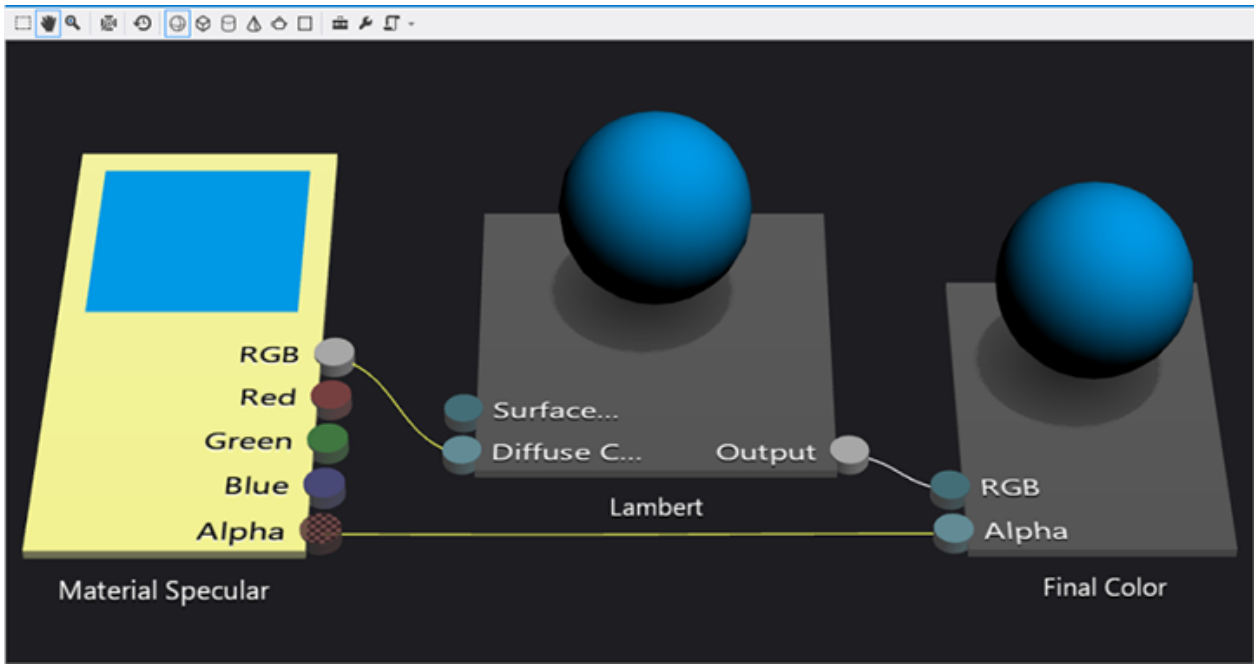
For more information, see Working with 3-D Models.

> **Note** The Model Editor is not intended to be a replacement for a full feature model editing app, but is appropriate for many simple viewing and editing scenarios.

Shader Designer

Use the Shader Designer to create custom visual effects for your game or app even if you don't know HLSL programming.

You create a shader visually as a graph. Each node displays a preview of the output up to that operation. Here's an example that applies Lambert lighting with a sphere preview.



Use the Shader Editor to design, edit, and save shaders in the .dgsl format. It also exports the following formats.

- .hlsl (source code)
- .cso (bytecode)
- .h (HLSL bytecode array)

Create build customization files to convert any of these formats to .cso files at build time.

Here is a portion of HLSL code that is exported by the Shader Editor. This is only the code for the Lambert lighting node.

hlsl

```
//
// Lambert lighting function
//
float3 LambertLighting(
    float3 lightNormal,
    float3 surfaceNormal,
    float3 materialAmbient,
    float3 lightAmbient,
    float3 lightColor,
    float3 pixelColor
    )
{
    // Compute the amount of contribution per light.
    float diffuseAmount = saturate(dot(lightNormal, surfaceNormal));
    float3 diffuse = diffuseAmount * lightColor * pixelColor;

    // Combine ambient with diffuse.
```

```
    return saturate((materialAmbient * lightAmbient) + diffuse);
}
```

For more information, see Working with Shaders.

Build customizations for 3D assets

You can add build customizations to your project so that Visual Studio converts resources to usable formats. After that, you can load the assets into your app and use them by creating and filling DirectX resources just like you would in any other DirectX app.

To add a build customization, you right-click on the project in the **Solution Explorer** and select **Build Customizations...**. You can add the following types of build customizations to your project.

- Image Content Pipeline takes image files as input and outputs DirectDraw Surface (.dds) files.
- Mesh Content Pipeline takes mesh files (such as .fbx) and outputs .cmo mesh files.
- Shader Content Pipeline takes Visual Shader Graph (.dgsl) from the Visual Studio Shader Editor and outputs a Compiled Shader Output (.cso) file.

For more information, see Using 3-D Assets in Your Game or App.

Debugging DirectX graphics

Visual Studio provides graphics-specific debugging tools. Use these tools to debug things like:

- The graphics pipeline.
- The event call stack.
- The object table.
- The device state.
- Shader bugs.
- Uninitialized or incorrect constant buffers and parameters.
- DirectX version compatibility.
- Limited Direct2D support.
- Operating system and SDK requirements.

For more information, see Debugging DirectX Graphics.

Graphics diagnostics tools

Article • 03/02/2024

The graphics diagnostic tools are available from within Windows 10 as an optional feature. To use the graphics diagnostic features (provided in the runtime and Visual Studio) to develop DirectX apps or games, add the optional Graphics Tools feature.

1. Go to **Settings** > **System** > **Optional features** (if on a version older than Windows 10 22H2, navigate to **Settings** > **Apps** > **Apps & features** > **Optional features** instead).

2. If **Graphics Tools** is already listed under **Added features**, then you're done. Otherwise, click **Add a feature**.

3. Search for and/or select **Graphics Tools**, and then click **Add**.

Graphics diagnostics features include the ability to create Direct3D debug devices (via Direct3D SDK Layers) in the DirectX runtime, plus Graphics Debugging, Frame Analysis, and GPU Usage.

- Graphics Debugging lets you trace the Direct3D calls being made by your app. Then, you can replay those calls, inspect parameters, debug and experiment with shaders, and visualize graphics assets to diagnose rendering issues. You can take logs on Windows PCs, simulators, or devices, and then play them back on different hardware.
- Graphics Frame Analysis in Visual Studio runs on a graphics debugging log, and gathers baseline timing for the Direct3D draw calls. It then performs a set of experiments by modifying various graphics settings, and produces a table of timing results. You can use this data to understand graphics performance issues in your app, and you can review results of the various experiments to identify opportunities for performance improvements.
- GPU Usage in Visual Studio allows you to monitor GPU use in real time. It collects and analyzes the timing data of the workloads being handled by the CPU and GPU, so that you can determine where any bottlenecks are.

Related topics

Graphics Diagnostics Overview in Visual Studio

Feedback

Was this page helpful? Yes No

Provide product feedback | Get help at Microsoft Q&A

Launching and resuming apps (DirectX and C++)

Article • 10/20/2022

Learn how to launch, suspend, and resume your Universal Windows Platform (UWP) DirectX app.

Topic	Description
How to activate an app	This topic shows how to define the activation experience for a UWP DirectX app.
How to suspend an app	This topic shows how to save important system state and app data when the system suspends your UWP DirectX app.
How to resume an app	This topic shows how to restore important application data when the system resumes your UWP DirectX app.

How to activate an app (DirectX and C++)

Article • 10/20/2022

This topic shows how to define the activation experience for a Universal Windows Platform (UWP) DirectX app.

Register the app activation event handler

First, register to handle the **CoreApplicationView::Activated** event, which is raised when your app is started and initialized by the operating system.

Add this code to your implementation of the **IFrameworkView::Initialize** method of your view provider (named **MyViewProvider** in the example):

C++

```
void App::Initialize(CoreApplicationView^ applicationView)
{
    // Register event handlers for the app lifecycle. This example includes
Activated, so that we
    // can make the CoreWindow active and start rendering on the window.
    applicationView->Activated +=
        ref new TypedEventHandler<CoreApplicationView^,
IActivatedEventArgs^>(this, &App::OnActivated);

  //...

}
```

Activate the CoreWindow instance for the app

When your app starts, you must obtain a reference to the **CoreWindow** for your app. **CoreWindow** contains the window event message dispatcher that your app uses to process window events. Obtain this reference in your callback for the app activation event by calling **CoreWindow::GetForCurrentThread**. Once you have obtained this reference, activate the main app window by calling **CoreWindow::Activate**.

C++

```
void App::OnActivated(CoreApplicationView^ applicationView,
IActivatedEventArgs^ args)
{
```

```
    // Run() won't start until the CoreWindow is activated.
    CoreWindow::GetForCurrentThread()->Activate();
}
```

Start processing event message for the main app window

Your callbacks occur as event messages are processed by the CoreDispatcher for the app's CoreWindow. This callback will not be invoked if you do not call CoreDispatcher::ProcessEvents from your app's main loop (implemented in the IFrameworkView::Run method of your view provider).

syntax

```
// This method is called after the window becomes active.
void App::Run()
{
    while (!m_windowClosed)
    {
        if (m_windowVisible)
        {
            CoreWindow::GetForCurrentThread()->Dispatcher-
>ProcessEvents(CoreProcessEventsOption::ProcessAllIfPresent);

            m_main->Update();

            if (m_main->Render())
            {
                m_deviceResources->Present();
            }
        }
        else
        {
            CoreWindow::GetForCurrentThread()->Dispatcher-
>ProcessEvents(CoreProcessEventsOption::ProcessOneAndAllPending);
        }
    }
}
```

Related topics

- How to suspend an app (DirectX and C++)
- How to resume an app (DirectX and C++)

How to suspend an app (DirectX and C++)

Article • 10/20/2022

This topic shows how to save important system state and app data when the system suspends your Universal Windows Platform (UWP) DirectX app.

Register the suspending event handler

First, register to handle the CoreApplication::Suspending event, which is raised when your app is moved to a suspended state by a user or system action.

Add this code to your implementation of the IFrameworkView::Initialize method of your view provider:

```
C++
```

```
void App::Initialize(CoreApplicationView^ applicationView)
{
  //...

    CoreApplication::Suspending +=
        ref new EventHandler<SuspendingEventArgs^>(this,
&App::OnSuspending);

  //...
}
```

Save any app data before suspending

When your app handles the CoreApplication::Suspending event, it has the opportunity to save its important application data in the handler function. The app should use the LocalSettings storage API to save simple application data synchronously. If you are developing a game, save any critical game state information. Don't forget to suspend the audio processing!

Now, implement the callback. Save the app data in this method.

```
C++
```

```
void App::OnSuspending(Platform::Object^ sender, SuspendingEventArgs^ args)
{
```

```
    // Save app state asynchronously after requesting a deferral. Holding a
deferral
    // indicates that the application is busy performing suspending
operations. Be
    // aware that a deferral may not be held indefinitely. After about five
seconds,
    // the app will be forced to exit.
    SuspendingDeferral^ deferral = args->SuspendingOperation->GetDeferral();

    create_task([this, deferral]()
    {
        m_deviceResources->Trim();

        // Insert your code here.

        deferral->Complete();
    });
}
```

This callback must complete with 5 seconds. During this callback, you must request a deferral by calling **SuspendingOperation::GetDeferral**, which starts the countdown. When your app completes the save operation, call **SuspendingDeferral::Complete** to tell the system that your app is now ready to be suspended. If you do not request a deferral, or if your app takes longer than 5 seconds to save the data, your app is automatically suspended.

This callback occurs as an event message processed by the **CoreDispatcher** for the app's **CoreWindow**. This callback will not be invoked if you do not call **CoreDispatcher::ProcessEvents** from your app's main loop (implemented in the **IFrameworkView::Run** method of your view provider).

syntax

```
// This method is called after the window becomes active.
void App::Run()
{
    while (!m_windowClosed)
    {
        if (m_windowVisible)
        {
            CoreWindow::GetForCurrentThread()->Dispatcher-
>ProcessEvents(CoreProcessEventsOption::ProcessAllIfPresent);

            m_main->Update();

            if (m_main->Render())
            {
                m_deviceResources->Present();
            }
        }
```

```
        else
        {
            CoreWindow::GetForCurrentThread()->Dispatcher-
>ProcessEvents(CoreProcessEventsOption::ProcessOneAndAllPending);
        }
    }
}
```

Call Trim()

Starting in Windows 8.1, all DirectX UWP apps must call **IDXGIDevice3::Trim** when suspending. This call tells the graphics driver to release all temporary buffers allocated for the app, which reduces the chance that the app will be terminated to reclaim memory resources while in the suspend state. This is a certification requirement for Windows 8.1.

C++

```
void App::OnSuspending(Platform::Object^ sender, SuspendingEventArgs^ args)
{
    // Save app state asynchronously after requesting a deferral. Holding a
deferral
    // indicates that the application is busy performing suspending
operations. Be
    // aware that a deferral may not be held indefinitely. After about five
seconds,
    // the app will be forced to exit.
    SuspendingDeferral^ deferral = args->SuspendingOperation->GetDeferral();

    create_task([this, deferral]()
    {
        m_deviceResources->Trim();

        // Insert your code here.

        deferral->Complete();
    });
}

// Call this method when the app suspends. It provides a hint to the driver
that the app
// is entering an idle state and that temporary buffers can be reclaimed for
use by other apps.
void DX::DeviceResources::Trim()
{
    ComPtr<IDXGIDevice3> dxgiDevice;
    m_d3dDevice.As(&dxgiDevice);

    dxgiDevice->Trim();
}
```

Release any exclusive resources and file handles

When your app handles the **CoreApplication::Suspending** event, it also has the opportunity to release exclusive resources and file handles. Explicitly releasing exclusive resources and file handles helps to ensure that other apps can access them while your app isn't using them. When the app is activated after termination, it should open its exclusive resources and file handles.

Remarks

The system suspends your app whenever the user switches to another app or to the desktop. The system resumes your app whenever the user switches back to it. When the system resumes your app, the content of your variables and data structures is the same as it was before the system suspended the app. The system restores the app exactly where it left off, so that it appears to the user as if it's been running in the background.

The system attempts to keep your app and its data in memory while it's suspended. However, if the system does not have the resources to keep your app in memory, the system will terminate your app. When the user switches back to a suspended app that has been terminated, the system sends an **Activated** event and should restore its application data in its handler for the **CoreApplicationView::Activated** event.

The system doesn't notify an app when it's terminated, so your app must save its application data and release exclusive resources and file handles when it's suspended, and restore them when the app is activated after termination.

Related topics

- How to resume an app (DirectX and C++)
- How to activate an app (DirectX and C++)

How to resume an app (DirectX and C++)

Article • 10/20/2022

This topic shows how to restore important application data when the system resumes your Universal Windows Platform (UWP) DirectX app.

Register the resuming event handler

Register to handle the CoreApplication::Resuming event, which indicates that the user switched away from your app and then back to it.

Add this code to your implementation of the IFrameworkView::Initialize method of your view provider:

C++

```
// The first method is called when the IFrameworkView is being created.
void App::Initialize(CoreApplicationView^ applicationView)
{
  //...

    CoreApplication::Resuming +=
        ref new EventHandler<Platform::Object^>(this, &App::OnResuming);

  //...

}
```

Refresh displayed content after suspension

When your app handles the Resuming event, it has the opportunity to refresh its displayed content. Restore any app you have saved with your handler for CoreApplication::Suspending, and restart processing. Game devs: if you've suspended your audio engine, now's the time to restart it.

C++

```
void App::OnResuming(Platform::Object^ sender, Platform::Object^ args)
{
    // Restore any data or state that was unloaded on suspend. By default,
data
    // and state are persisted when resuming from suspend. Note that this
```

```
event
    // does not occur if the app was previously terminated.

    // Insert your code here.
}
```

This callback occurs as an event message processed by the CoreDispatcher for the app's CoreWindow. This callback will not be invoked if you do not call CoreDispatcher::ProcessEvents from your app's main loop (implemented in the IFrameworkView::Run method of your view provider).

syntax

```
// This method is called after the window becomes active.
void App::Run()
{
    while (!m_windowClosed)
    {
        if (m_windowVisible)
        {
            CoreWindow::GetForCurrentThread()->Dispatcher-
>ProcessEvents(CoreProcessEventsOption::ProcessAllIfPresent);

            m_main->Update();

            if (m_main->Render())
            {
                m_deviceResources->Present();
            }
        }
        else
        {
            CoreWindow::GetForCurrentThread()->Dispatcher-
>ProcessEvents(CoreProcessEventsOption::ProcessOneAndAllPending);
        }
    }
}
```

Remarks

The system suspends your app whenever the user switches to another app or to the desktop. The system resumes your app whenever the user switches back to it. When the system resumes your app, the content of your variables and data structures is the same as it was before the system suspended the app. The system restores the app exactly where it left off, so that it appears to the user as if it's been running in the background. However, the app may have been suspended for a significant amount of time, so it should refresh any displayed content that might have changed while the app was

suspended, and restart any rendering or audio processing threads. If you've saved any game state data during a previous suspend event, restore it now.

Related topics

- How to suspend an app (DirectX and C++)
- How to activate an app (DirectX and C++)

DirectX Samples

Article • 10/20/2022

These are some game samples developed with DirectX.

Topic	Description
Create a simple UWP game with DirectX	In this set of tutorials, you'll learn how to use DirectX and C++/WinRT to create the basic Universal Windows Platform (UWP) sample game named **Simple3DGameDX**. This set of tutorials focus on key UWP DirectX game development techniques and considerations.
Developing *Marble Maze*—a Universal Windows Platform (UWP) game built with C++ for DirectX	Create a 3D game that works on various types of devices such as tablets, desktop PCs, and laptops.

Create a simple Universal Windows Platform (UWP) game with DirectX

Article • 10/20/2022

In this set of tutorials, you'll learn how to use DirectX and C++/WinRT to create the basic Universal Windows Platform (UWP) sample game named **Simple3DGameDX**. The gameplay takes place in a simple first-person 3D shooting gallery.

> **Note**
>
> The link from which you can download the **Simple3DGameDX** sample game itself is **Direct3D sample game**. The C++/WinRT source code is in the folder named `cppwinrt`. For info about other UWP sample apps, see **Sample applications for Windows development**.

These tutorials cover all of the major parts of a game, including the processes for loading assets such as arts and meshes, creating a main game loop, implementing a simple rendering pipeline, and adding sound and controls.

You'll also see UWP game development techniques and considerations. We'll focus on key UWP DirectX game development concepts, and call out Windows-Runtime-specific considerations around those concepts.

Objective

To learn about the basic concepts and components of a UWP DirectX game, and to become more comfortable designing UWP games with DirectX.

What you need to know

For this tutorial, you need to be familiar with these subjects.

- C++/WinRT. C++/WinRT is a standard modern C++17 language projection for Windows APIs, implemented as a header-file-based library, and designed to provide you with first-class access to the modern Windows APIs.
- Basic linear algebra and Newtonian physics concepts.
- Basic graphics programming terminology.
- Basic Windows programming concepts.
- Basic familiarity with the Direct2D and Direct3D 11 APIs.

Direct3D UWP shooting gallery sample

The **Simple3DGameDX** sample game implements a simple first-person 3D shooting gallery, where the player fires balls at moving targets. Hitting each target awards a set number of points, and the player can progress through 6 levels of increasing challenge. At the end of the levels, the points are tallied, and the player is awarded a final score.

The sample demonstrates these game concepts.

- Interoperation between DirectX 11.1 and the Windows Runtime
- A first-person 3D perspective and camera
- Stereoscopic 3D effects
- Collision-detection between objects in 3D
- Handling player input for mouse, touch, and Xbox controller controls
- Audio mixing and playback
- A basic game state-machine

Topic	Description
Set up the game project	The first step in developing your game is to set up a project in Microsoft Visual Studio. After you've configured a project specifically for game development, you could later re-use it as a kind of template.
Define the game's UWP app framework	The first step in coding a Universal Windows Platform (UWP) game is building the framework that lets the app object interact with Windows.
Game flow management	Define the high-level state machine to enable player and system interaction. Learn how UI interacts with the overall game's state machine and how to create event handlers for UWP games.

Topic	Description
Define the main game object	Now, we look at the details of the sample game's main object and how the rules it implements translate into interactions with the game world.
Rendering framework I: Intro to rendering	Learn how to develop the rendering pipeline to display graphics. Intro to rendering.
Rendering framework II: Game rendering	Learn how to assemble the rendering pipeline to display graphics. Game rendering, set up and prepare data.
Add a user interface	Learn how to add a 2D user interface overlay to a DirectX UWP game.
Add controls	Now, we take a look at how the sample game implements move-look controls in a 3-D game, and how to develop basic touch, mouse, and game controller controls.
Add sound	Develop a simple sound engine using XAudio2 APIs to playback game music and sound effects.
Extend the sample game	Learn how to implement a XAML overlay for a UWP DirectX game.

Set up the game project

Article • 10/20/2022

> **Note**
>
> This topic is part of the **Create a simple Universal Windows Platform (UWP) game with DirectX** tutorial series. The topic at that link sets the context for the series.

The first step in developing your game is to create a project in Microsoft Visual Studio. After you've configured a project specifically for game development, you could later re-use it as a kind of template.

Objectives

- Create a new project in Visual Studio using a project template.
- Understand the game's entry point and initialization by examining the source file for the **App** class.
- Look at the game loop.
- Review the project's **package.appxmanifest** file.

Create a new project in Visual Studio

> **Note**
>
> For info about setting up Visual Studio for C++/WinRT development—including installing and using the C++/WinRT Visual Studio Extension (VSIX) and the NuGet package (which together provide project template and build support)—see **Visual Studio support for C++/WinRT**.

First install (or update to) the latest version of the C++/WinRT Visual Studio Extension (VSIX); see the note above. Then, in Visual Studio, create a new project based on the **Core App (C++/WinRT)** project template. Target the latest generally-available (that is, not preview) version of the Windows SDK.

Review the App class to understand IFrameworkViewSource and IFrameworkView

In your Core App project, open the source code file `App.cpp`. In there is the implementation of the **App** class, which represents the app and its lifecycle. In this case, of course, we know that the app is a game. But we'll refer to it as an *app* in order to talk more generally about how a Universal Windows Platform (UWP) app initializes.

The wWinMain function

The **wWinMain** function is the entry point for the app. Here's what **wWinMain** looks like (from `App.cpp`).

C++/WinRT

```
int __stdcall wWinMain(HINSTANCE, HINSTANCE, PWSTR, int)
{
    CoreApplication::Run(winrt::make<App>());
}
```

We make an instance of the **App** class (this is the one, and only, instance of **App** that's created), and we pass that to the static **CoreApplication.Run** method. Note that **CoreApplication.Run** expects an **IFrameworkViewSource** interface. So the **App** class needs to implement that interface.

The next two sections in this topic describe the **IFrameworkViewSource** and **IFrameworkView** interfaces. Those interfaces (as well as **CoreApplication.Run**) represent a way for your app to supply Windows with a *view-provider*. Windows uses that view-provider to connect your app with the Windows shell so that you can handle application lifecycle events.

The IFrameworkViewSource interface

The **App** class does indeed implement **IFrameworkViewSource**, as you can see in the listing below.

C++/WinRT

```
struct App : winrt::implements<App, IFrameworkViewSource, IFrameworkView>
{
    ...
    IFrameworkView CreateView()
    {
        return *this;
    }
    ...
}
```

An object that implements **IFrameworkViewSource** is a *view-provider factory* object. That object's job is to manufacture and return a *view-provider* object.

IFrameworkViewSource has the single method IFrameworkViewSource::CreateView. Windows calls that function on the object that you pass to **CoreApplication.Run**. As you can see above, the **App::CreateView** implementation of that method returns `*this`. In other words, the **App** object returns itself. Since **IFrameworkViewSource::CreateView** has a return value type of IFrameworkView, it follows that the **App** class needs to implement *that* interface, too. And you can see that it does in the listing above.

The IFrameworkView interface

An object that implements IFrameworkView is a *view-provider* object. And we've now supplied Windows with that view-provider. It's that same **App** object that we created in **wWinMain**. So the **App** class serves as both *view-provider factory* and *view-provider*.

Now Windows can call the **App** class's implementations of the methods of **IFrameworkView**. In the implementations of those methods, your app has a chance to perform tasks such as initialization, to begin to load the resources it'll need, to connect up the appropriate event handlers, and to receive the CoreWindow that your app will use to display its output.

Your implementations of the methods of **IFrameworkView** are called in the order shown below.

- Initialize
- SetWindow
- Load
- The CoreApplicationView::Activated event is raised. So if you've (optionally) registered to handle that event, then your **OnActivated** handler is called at this time.
- Run
- Uninitialize

And here's the skeleton of the **App** class (in `App.cpp`), showing the signatures of those methods.

C++/WinRT

```
struct App : winrt::implements<App, IFrameworkViewSource, IFrameworkView>
{
    ...
    void Initialize(Windows::ApplicationModel::Core::CoreApplicationView
const& applicationView) { ... }
```

```
    void SetWindow(Windows::UI::Core::CoreWindow const& window) { ... }
    void Load(winrt::hstring const& entryPoint) { ... }
    void OnActivated(
        Windows::ApplicationModel::Core::CoreApplicationView const&
applicationView,
        Windows::ApplicationModel::Activation::IActivatedEventArgs const&
args) { ... }
    void Run() { ... }
    void Uninitialize() { ... }
    ...
}
```

This was just an introduction to **IFrameworkView**. We go into much more detail about these methods, and how to implement them, in Define the game's UWP app framework.

Tidy up the project

The Core App project that you created from the project template contains functionality that we should tidy up at this point. After that, we can use the project to recreate the shooting gallery game (**Simple3DGameDX**). Make the following changes to the **App** class in `App.cpp`.

- Delete its data members.
- Delete **OnPointerPressed**, **OnPointerMoved**, and **AddVisual**
- Delete the code from **SetWindow**.

The project will build and run, but it'll display only a solid color in the client area.

The game loop

To get an idea of what a game loop looks like, look in the source code for the Simple3DGameDX sample game that you downloaded.

The **App** class has a data member, named *m_main*, of type **GameMain**. And that member is used in **App::Run** like this.

C++/WinRT

```
void Run()
{
    m_main->Run();
}
```

You can find **GameMain::Run** in `GameMain.cpp`. It's the main loop of the game, and here's a very rough outline of it showing the most important features.

C++/WinRT

```
void GameMain::Run()
{
    while (!m_windowClosed)
    {
        if (m_visible)
        {

CoreWindow::GetForCurrentThread().Dispatcher().ProcessEvents(CoreProcessEven
tsOption::ProcessAllIfPresent);
            Update();
            m_renderer->Render();
            m_deviceResources->Present();
        }
        else
        {

CoreWindow::GetForCurrentThread().Dispatcher().ProcessEvents(CoreProcessEven
tsOption::ProcessOneAndAllPending);
        }
    }
}
```

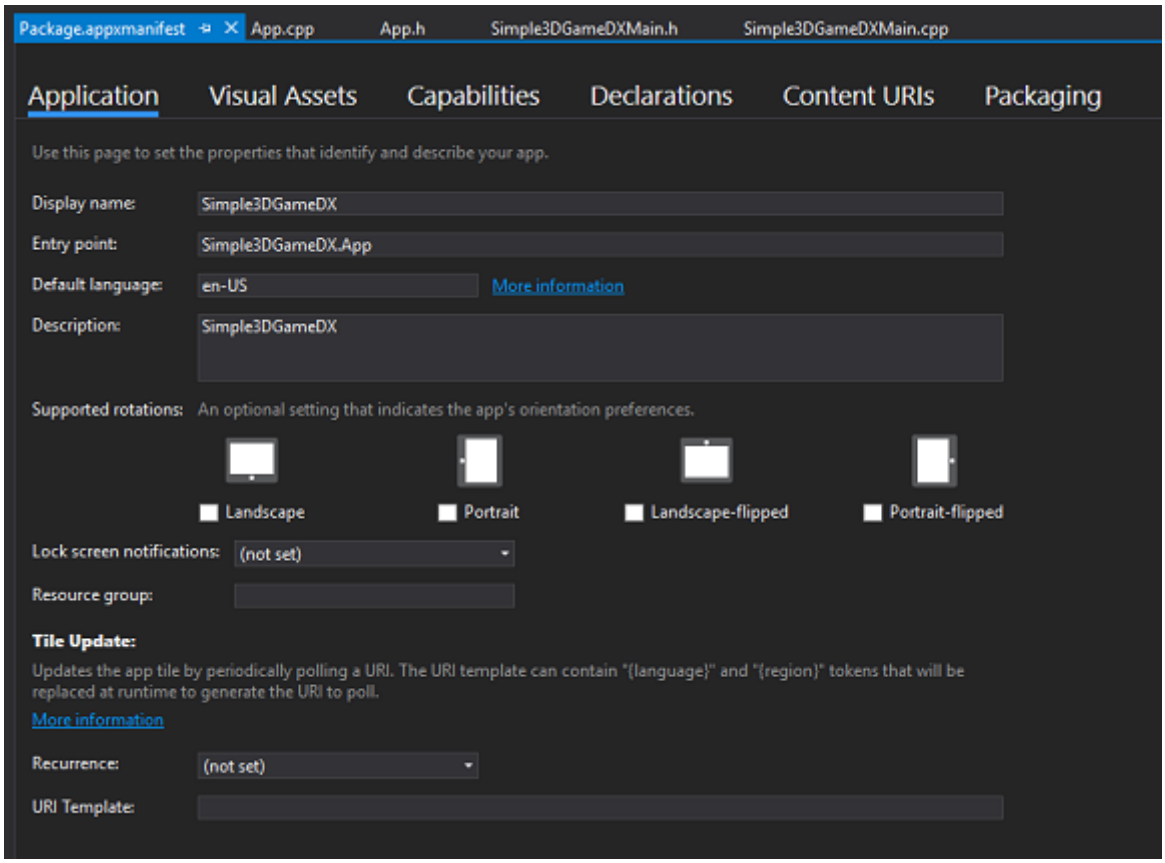
And here's a brief description of what this main game loop does.

If the window for your game isn't closed, dispatch all events, update the timer, and then render and present the results of the graphics pipeline. There's a lot more to say about those concerns, and we'll do that in the topics Define the game's UWP app framework, Rendering framework I: Intro to rendering, and Rendering framework II: Game rendering. But this is the basic code structure of a UWP DirectX game.

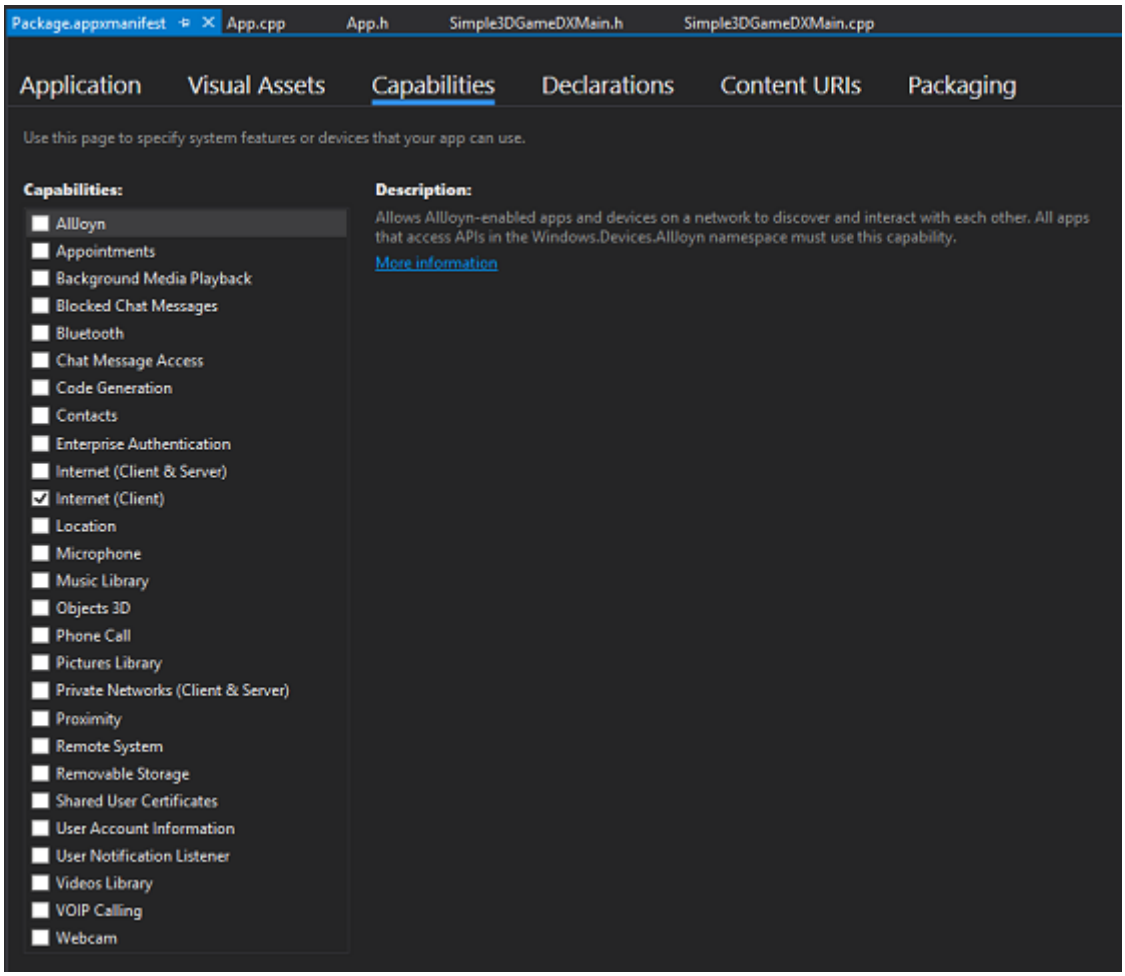
Review and update the package.appxmanifest file

The **Package.appxmanifest** file contains metadata about a UWP project. Those metadata are used for packaging and launching your game, and for submission to the Microsoft Store. The file also contains important info that the player's system uses to provide access to the system resources that the game needs to run.

Launch the **manifest designer** by double-clicking the **Package.appxmanifest** file in **Solution Explorer**.



For more info about the **package.appxmanifest** file and packaging, see Manifest Designer. For now, take a look at the **Capabilities** tab, and look at the options provided.



If you don't select the capabilities that your game uses, such as access to the **Internet** for global high score board, then you won't be able to access the corresponding resources nor features. When you create a new game, make sure that you select any capabilities needed by APIs that your game calls.

Now let's look at the rest of the files that come with the **Simple3DGameDX** sample game.

Review other important libraries and source code files

If you do intend to create a kind of game project template for yourself, so that you can re-use that as a starting-point for future projects, then you'll want to copy `GameMain.h` and `GameMain.cpp` out of the Simple3DGameDX project that you downloaded, and add those to your new Core App project. Study those files, learn what they do, and remove anything that's specific to **Simple3DGameDX**. Also comment out anything that depends on code that you've not yet copied. Just by way of an example, `GameMain.h` depends on `GameRenderer.h`. You'll be able to uncomment as you copy more files out of **Simple3DGameDX**.

Here's a brief survey of some of the files in **Simple3DGameDX** that you'll find useful to include in your template, if you're making one. In any case, these are equally important to understanding how **Simple3DGameDX** itself work.

Source file	File folder	Description
DeviceResources.h/.cpp	Utilities	Defines the **DeviceResources** class, which controls all DirectX device resources. Also defines the **IDeviceNotify** interface, used to notify your application that the graphics adapter device has been lost or re-created.
DirectXSample.h	Utilities	Implements helper functions such as **ConvertDipsToPixels**. **ConvertDipsToPixels** converts a length in device-independent pixels (DIPs) to a length in physical pixels.
GameTimer.h/.cpp	Utilities	Defines a high-resolution timer useful for gaming or interactive rendering apps.
GameRenderer.h/.cpp	Rendering	Defines the **GameRenderer** class, which implements a basic rendering pipeline.
GameHud.h/.cpp	Rendering	Defines a class to render a heads up display (HUD) for the game, using Direct2D and DirectWrite.

Source file	File folder	Description
VertexShader.hlsl and VertexShaderFlat.hlsl	Shaders	Contains the high-level shader language (HLSL) code for basic vertex shaders.
PixelShader.hlsl and PixelShaderFlat.hlsl	Shaders	Contains the high-level shader language (HLSL) code for basic pixel shaders.
ConstantBuffers.hlsli	Shaders	Contains data structure definitions for constant buffers and shader structures used to pass model-view-projection (MVP) matrices and per-vertex data to the vertex shader.
pch.h/.cpp	N/A	Contains common C++/WinRT, Windows, and DirectX includes.

Next steps

At this point, we've shown how to create a new UWP project for a DirectX game, looked at some of the pieces in it, and started thinking about how to turn that project into a kind of re-usable template for games. We've also looked at some of the important pieces of the **Simple3DGameDX** sample game.

The next section is Define the game's UWP app framework. There we'll look more closely at how **Simple3DGameDX** works.

Define the game's UWP app framework

Article • 10/20/2022

> ⓘ Note
>
> This topic is part of the **Create a simple Universal Windows Platform (UWP) game with DirectX** tutorial series. The topic at that link sets the context for the series.

The first step in coding a Universal Windows Platform (UWP) game is building the framework that lets the app object interact with Windows, including Windows Runtime features such as suspend-resume event handling, changes in window visibility, and snapping.

Objectives

- Set up the framework for a Universal Windows Platform (UWP) DirectX game, and implement the state machine that defines the overall game flow.

> ⓘ Note
>
> To follow along with this topic, look in the source code for the **Simple3DGameDX** sample game that you downloaded.

Introduction

In the Set up the game project topic, we introduced the **wWinMain** function as well as the **IFrameworkViewSource** and **IFrameworkView** interfaces. We learned that the **App** class (which you can see defined in the `App.cpp` source code file in the **Simple3DGameDX** project) serves as both *view-provider factory* and *view-provider*.

This topic picks up from there, and goes into much more detail about how the **App** class in a game should implement the methods of **IFrameworkView**.

The App::Initialize method

Upon application launch, the first method that Windows calls is our implementation of **IFrameworkView::Initialize**.

Your implementation should handle the most fundamental behaviors of a UWP game, such as making sure that the game can handle a suspend (and a possible later resume) event by subscribing to those events. We also have access to the display adapter device here, so we can create graphics resources that depend on the device.

C++/WinRT

```
void Initialize(CoreApplicationView const& applicationView)
{
    applicationView.Activated({ this, &App::OnActivated });

    CoreApplication::Suspending({ this, &App::OnSuspending });

    CoreApplication::Resuming({ this, &App::OnResuming });

    // At this point we have access to the device.
    // We can create the device-dependent resources.
    m_deviceResources = std::make_shared<DX::DeviceResources>();
}
```

Avoid raw pointers whenever possible (and it's nearly always possible).

- For Windows Runtime types, you can very often avoid pointers altogether and just construct a value on the stack. If you do need a pointer, then use **winrt::com_ptr** (we'll see an example of that soon).
- For unique pointers, use **std::unique_ptr** and **std::make_unique**.
- For shared pointers, use **std::shared_ptr** and **std::make_shared**.

The App::SetWindow method

After **Initialize**, Windows calls our implementation of **IFrameworkView::SetWindow**, passing a **CoreWindow** object representing the game's main window.

In **App::SetWindow**, we subscribe to window-related events, and configure some window and display behaviors. For example, we construct a mouse pointer (via the **CoreCursor** class), which can be used by both mouse and touch controls. We also pass the window object to our device-dependent resources object.

We'll talk more about handling events in the Game flow management topic.

C++/WinRT

```
void SetWindow(CoreWindow const& window)
{
    //CoreWindow window = CoreWindow::GetForCurrentThread();
    window.Activate();
```

```
    window.PointerCursor(CoreCursor(CoreCursorType::Arrow, 0));

    PointerVisualizationSettings visualizationSettings{
PointerVisualizationSettings::GetForCurrentView() };
    visualizationSettings.IsContactFeedbackEnabled(false);
    visualizationSettings.IsBarrelButtonFeedbackEnabled(false);

    m_deviceResources->SetWindow(window);

    window.Activated({ this, &App::OnWindowActivationChanged });

    window.SizeChanged({ this, &App::OnWindowSizeChanged });

    window.Closed({ this, &App::OnWindowClosed });

    window.VisibilityChanged({ this, &App::OnVisibilityChanged });

    DisplayInformation currentDisplayInformation{
DisplayInformation::GetForCurrentView() };

    currentDisplayInformation.DpiChanged({ this, &App::OnDpiChanged });

    currentDisplayInformation.OrientationChanged({ this,
&App::OnOrientationChanged });

    currentDisplayInformation.StereoEnabledChanged({ this,
&App::OnStereoEnabledChanged });

    DisplayInformation::DisplayContentsInvalidated({ this,
&App::OnDisplayContentsInvalidated });
}
```

The App::Load method

Now that the main window is set, our implementation of **IFrameworkView::Load** is called. **Load** is a better place to pre-fetch game data or assets than **Initialize** and **SetWindow**.

C++/WinRT

```
void Load(winrt::hstring const& /* entryPoint */)
{
    if (!m_main)
    {
        m_main = winrt::make_self<GameMain>(m_deviceResources);
    }
}
```

As you can see, the actual work is delegated to the constructor of the **GameMain** object that we make here. The **GameMain** class is defined in `GameMain.h` and `GameMain.cpp`.

The GameMain::GameMain constructor

The **GameMain** constructor (and the other member functions that it calls) begins a set of asynchronous loading operations to create the game objects, load graphics resources, and initialize the game's state machine. We also do any necessary preparations before the game begins, such as setting any starting states or global values.

Windows imposes a limit on the time your game can take before it begins processing input. So using async, as we do here, means that **Load** can return quickly while the work that it has begun continues in the background. If loading takes a long time, or if there are lots of resources, then providing your users with a frequently updated progress bar is a good idea.

If you're new to asynchronous programming, then see Concurrency and asynchronous operations with C++/WinRT.

C++/WinRT

```
GameMain::GameMain(std::shared_ptr<DX::DeviceResources> const&
deviceResources) :
    m_deviceResources(deviceResources),
    m_windowClosed(false),
    m_haveFocus(false),
    m_gameInfoOverlayCommand(GameInfoOverlayCommand::None),
    m_visible(true),
    m_loadingCount(0),
    m_updateState(UpdateEngineState::WaitingForResources)
{
    m_deviceResources->RegisterDeviceNotify(this);

    m_renderer = std::make_shared<GameRenderer>(m_deviceResources);
    m_game = std::make_shared<Simple3DGame>();

    m_uiControl = m_renderer->GameUIControl();

    m_controller = std::make_shared<MoveLookController>
(CoreWindow::GetForCurrentThread());

    auto bounds = m_deviceResources->GetLogicalSize();

    m_controller->SetMoveRect(
        XMFLOAT2(0.0f, bounds.Height - GameUIConstants::TouchRectangleSize),
        XMFLOAT2(GameUIConstants::TouchRectangleSize, bounds.Height)
        );
    m_controller->SetFireRect(
```

```
        XMFLOAT2(bounds.Width - GameUIConstants::TouchRectangleSize,
bounds.Height - GameUIConstants::TouchRectangleSize),
        XMFLOAT2(bounds.Width, bounds.Height)
        );

    SetGameInfoOverlay(GameInfoOverlayState::Loading);
    m_uiControl->SetAction(GameInfoOverlayCommand::None);
    m_uiControl->ShowGameInfoOverlay();

    // Asynchronously initialize the game class and load the renderer device
resources.
    // By doing all this asynchronously, the game gets to its main loop more
quickly
    // and in parallel all the necessary resources are loaded on other
threads.
    ConstructInBackground();
}

winrt::fire_and_forget GameMain::ConstructInBackground()
{
    auto lifetime = get_strong();

    m_game->Initialize(m_controller, m_renderer);

    co_await m_renderer->CreateGameDeviceResourcesAsync(m_game);

    // The finalize code needs to run in the same thread context
    // as the m_renderer object was created because the D3D device context
    // can ONLY be accessed on a single thread.
    // co_await of an IAsyncAction resumes in the same thread context.
    m_renderer->FinalizeCreateGameDeviceResources();

    InitializeGameState();

    if (m_updateState == UpdateEngineState::WaitingForResources)
    {
        // In the middle of a game so spin up the async task to load the
level.
        co_await m_game->LoadLevelAsync();

        // The m_game object may need to deal with D3D device context work
so
        // again the finalize code needs to run in the same thread
        // context as the m_renderer object was created because the D3D
        // device context can ONLY be accessed on a single thread.
        m_game->FinalizeLoadLevel();
        m_game->SetCurrentLevelToSavedState();
        m_updateState = UpdateEngineState::ResourcesLoaded;
    }
    else
    {
        // The game is not in the middle of a level so there aren't any
level
        // resources to load.
    }
```

```
    // Since Game loading is an async task, the app visual state
    // may be too small or not be activated. Put the state machine
    // into the correct state to reflect these cases.

    if (m_deviceResources->GetLogicalSize().Width <
GameUIConstants::MinPlayableWidth)
    {
        m_updateStateNext = m_updateState;
        m_updateState = UpdateEngineState::TooSmall;
        m_controller->Active(false);
        m_uiControl->HideGameInfoOverlay();
        m_uiControl->ShowTooSmall();
        m_renderNeeded = true;
    }
    else if (!m_haveFocus)
    {
        m_updateStateNext = m_updateState;
        m_updateState = UpdateEngineState::Deactivated;
        m_controller->Active(false);
        m_uiControl->SetAction(GameInfoOverlayCommand::None);
        m_renderNeeded = true;
    }
}

void GameMain::InitializeGameState()
{
    // Set up the initial state machine for handling Game playing state.
    ...
}
```

Here's an outline of the sequence of work that's kicked off by the constructor.

- Create and initialize an object of type **GameRenderer**. For more information, see Rendering framework I: Intro to rendering.
- Create and initialize an object of type **Simple3DGame**. For more information, see Define the main game object.
- Create the game UI control object, and display game info overlay to show a progress bar as the resource files load. For more information, see Adding a user interface.
- Create a controller object to read input from the controller (touch, mouse, or game controller). For more information, see Adding controls.
- Define two rectangular areas in the lower-left and lower-right corners of the screen for the move and camera touch controls, respectively. The player uses the lower-left rectangle (defined in the call to **SetMoveRect**) as a virtual control pad for moving the camera forward and backward, and side to side. The lower-right rectangle (defined by the **SetFireRect** method) is used as a virtual button to fire the ammo.

- Use coroutines to break resource loading into separate stages. Access to the Direct3D device context is restricted to the thread on which the device context was created; while access to the Direct3D device for object creation is free-threaded. Consequently, the **GameRenderer::CreateGameDeviceResourcesAsync** coroutine can run on a separate thread from the completion task (**GameRenderer::FinalizeCreateGameDeviceResources**), which runs on the original thread.
- We use a similar pattern for loading level resources with **Simple3DGame::LoadLevelAsync** and **Simple3DGame::FinalizeLoadLevel**.

We'll see more of **GameMain::InitializeGameState** in the next topic (Game flow management).

The App::OnActivated method

Next, the **CoreApplicationView::Activated** event is raised. So any **OnActivated** event handler that you have (such as our **App::OnActivated** method) is called.

C++/WinRT

```
void OnActivated(CoreApplicationView const& /* applicationView */,
IActivatedEventArgs const& /* args */)
{
    CoreWindow window = CoreWindow::GetForCurrentThread();
    window.Activate();
}
```

The only work we do here is to activate the main **CoreWindow**. Alternatively, you can choose to do that in **App::SetWindow**.

The App::Run method

Initialize, **SetWindow**, and **Load** have set the stage. Now that the game is up and running, our implementation of **IFrameworkView::Run** is called.

C++/WinRT

```
void Run()
{
    m_main->Run();
}
```

Again, work is delegated to **GameMain**.

The GameMain::Run method

GameMain::Run is the main loop of the game; you can find it in `GameMain.cpp`. The basic logic is that while the window for your game remains open, dispatch all events, update the timer, and then render and present the results of the graphics pipeline. Also here, the events used to transition between game states are dispatched and processed.

The code here is also concerned with two of the states in the game engine state machine.

- **UpdateEngineState::Deactivated**. This specifies that the game window is deactivated (has lost focus) or is snapped.
- **UpdateEngineState::TooSmall**. This specifies that the client area is too small to render the game in.

In either of these states, the game suspends event processing, and waits for the window to be activated, to unsnap, or to be resized.

While your game window is visible (Window.Visible is `true`), you must handle every event in the message queue as it arrives, and so you must call CoreWindowDispatch.ProcessEvents with the **ProcessAllIfPresent** option. Other options can cause delays in processing message events, which can make your game feel unresponsive, or result in touch behaviors that feel sluggish.

When the game is *not* visible (Window.Visible is `false`), or when it's suspended, or when it's too small (it's snapped), you don't want it to consume any resources cycling to dispatch messages that will never arrive. In this case, your game must use the **ProcessOneAndAllPending** option. That option blocks until it gets an event, and then processes that event (as well as any others that arrive in the process queue during the processing of the first). **CoreWindowDispatch.ProcessEvents** then immediately returns after the queue has been processed.

In the example code shown below, the *m_visible* data member represents the window's visibility. When the game is suspended, its window is not visible. When the window *is* visible, the value of *m_updateState* (an **UpdateEngineState** enum) determines further whether or not the window is deactivated (lost focus), too small (snapped), or the right size.

C++/WinRT

```
void GameMain::Run()
{
    while (!m_windowClosed)
    {
```

```
        if (m_visible)
        {
            switch (m_updateState)
            {
            case UpdateEngineState::Deactivated:
            case UpdateEngineState::TooSmall:
                if (m_updateStateNext ==
UpdateEngineState::WaitingForResources)
                {
                    WaitingForResourceLoading();
                    m_renderNeeded = true;
                }
                else if (m_updateStateNext ==
UpdateEngineState::ResourcesLoaded)
                {
                    // In the device lost case, we transition to the final
waiting state
                    // and make sure the display is updated.
                    switch (m_pressResult)
                    {
                    case PressResultState::LoadGame:
                        SetGameInfoOverlay(GameInfoOverlayState::GameStats);
                        break;

                    case PressResultState::PlayLevel:

SetGameInfoOverlay(GameInfoOverlayState::LevelStart);
                        break;

                    case PressResultState::ContinueLevel:
                        SetGameInfoOverlay(GameInfoOverlayState::Pause);
                        break;
                    }
                    m_updateStateNext = UpdateEngineState::WaitingForPress;
                    m_uiControl->ShowGameInfoOverlay();
                    m_renderNeeded = true;
                }

                if (!m_renderNeeded)
                {
                    // The App is not currently the active window and not in
a transient state so just wait for events.

CoreWindow::GetForCurrentThread().Dispatcher().ProcessEvents(CoreProcessEven
tsOption::ProcessOneAndAllPending);
                    break;
                }
                // otherwise fall through and do normal processing to get
the rendering handled.
            default:

CoreWindow::GetForCurrentThread().Dispatcher().ProcessEvents(CoreProcessEven
tsOption::ProcessAllIfPresent);
                Update();
                m_renderer->Render();
```

```
                m_deviceResources->Present();
                m_renderNeeded = false;
            }
        }
        else
        {

CoreWindow::GetForCurrentThread().Dispatcher().ProcessEvents(CoreProcessEven
tsOption::ProcessOneAndAllPending);
        }
    }
    m_game->OnSuspending();  // Exiting due to window close, so save state.
}
```

The App::Uninitialize method

When the game ends, our implementation of **IFrameworkView::Uninitialize** is called. This is our opportunity to perform cleanup. Closing the app window doesn't kill the app's process; but instead it writes the state of the app singleton to memory. If anything special must happen when the system reclaims this memory, including any special cleanup of resources, then put the code for that cleanup in **Uninitialize**.

In our case, **App::Uninitialize** is a no-op.

C++

```
void Uninitialize()
{
}
```

Tips

When developing your own game, design your startup code around the methods described in this topic. Here's a simple list of basic suggestions for each method.

- Use **Initialize** to allocate your main classes, and connect up the basic event handlers.
- Use **SetWindow** to subscribe to any window-specific events, and to pass your main window to your device-dependent resources object so that it can use that window when creating a swap chain.
- Use **Load** to handle any remaining setup, and to initiate the asynchronous creation of objects, and loading of resources. If you need to create any temporary files or data, such as procedurally generated assets, then do that here, too.

Next steps

This topic has covered some of the basic structure of a UWP game that uses DirectX. It's a good idea to keep these methods in mind, because we'll be referring back to some of them in later topics.

In the next topic—Game flow management—we'll take an in-depth look at how to manage game states and event handling in order to keep the game flowing.

Game flow management

Article • 10/20/2022

> **Note**
>
> This topic is part of the **Create a simple Universal Windows Platform (UWP) game with DirectX** tutorial series. The topic at that link sets the context for the series.

The game now has a window, has registered some event handlers, and has loaded assets asynchronously. This topic explains the use of game states, how to manage specific key game states, and how to create an update loop for the game engine. Then we'll learn about the user interface flow and, finally, understand more about the event handlers that are needed for a UWP game.

Game states used to manage game flow

We make use of game states to manage the flow of the game.

When the **Simple3DGameDX** sample game runs for the very first time on a machine, it's in a state where no game has been started. Subsequent times the game runs, it can be in any of these states.

- No game has been started, or the game is between levels (the high score is zero).
- The game loop is running, and is in the middle of a level.
- The game loop is not running due to a game having been completed (the high score has a non-zero value).

Your game can have as many states as it needs. But remember that it can be terminated at any time. And when it resumes, the user expects it to resume in the state it was in when it was terminated.

Game state management

So, during game initialization, you'll need to support cold-starting the game as well as resuming the game after stopping it in flight. The **Simple3DGameDX** sample always saves its game state in order to give the impression that it never stopped.

In response to a suspend event, gameplay is suspended, but the resources of the game are still in memory. Likewise, the resume event is handled to ensure that the sample

game picks up in the state it was in when it was suspended or terminated. Depending on the state, different options are presented to the player.

- If the game resumes mid-level, then it appears paused, and the overlay offers the option to continue.
- If the game resumes in a state where the game is completed, then it displays the high scores and an option to play a new game.
- Lastly, if the game resumes before a level has started, then the overlay presents a start option to the user.

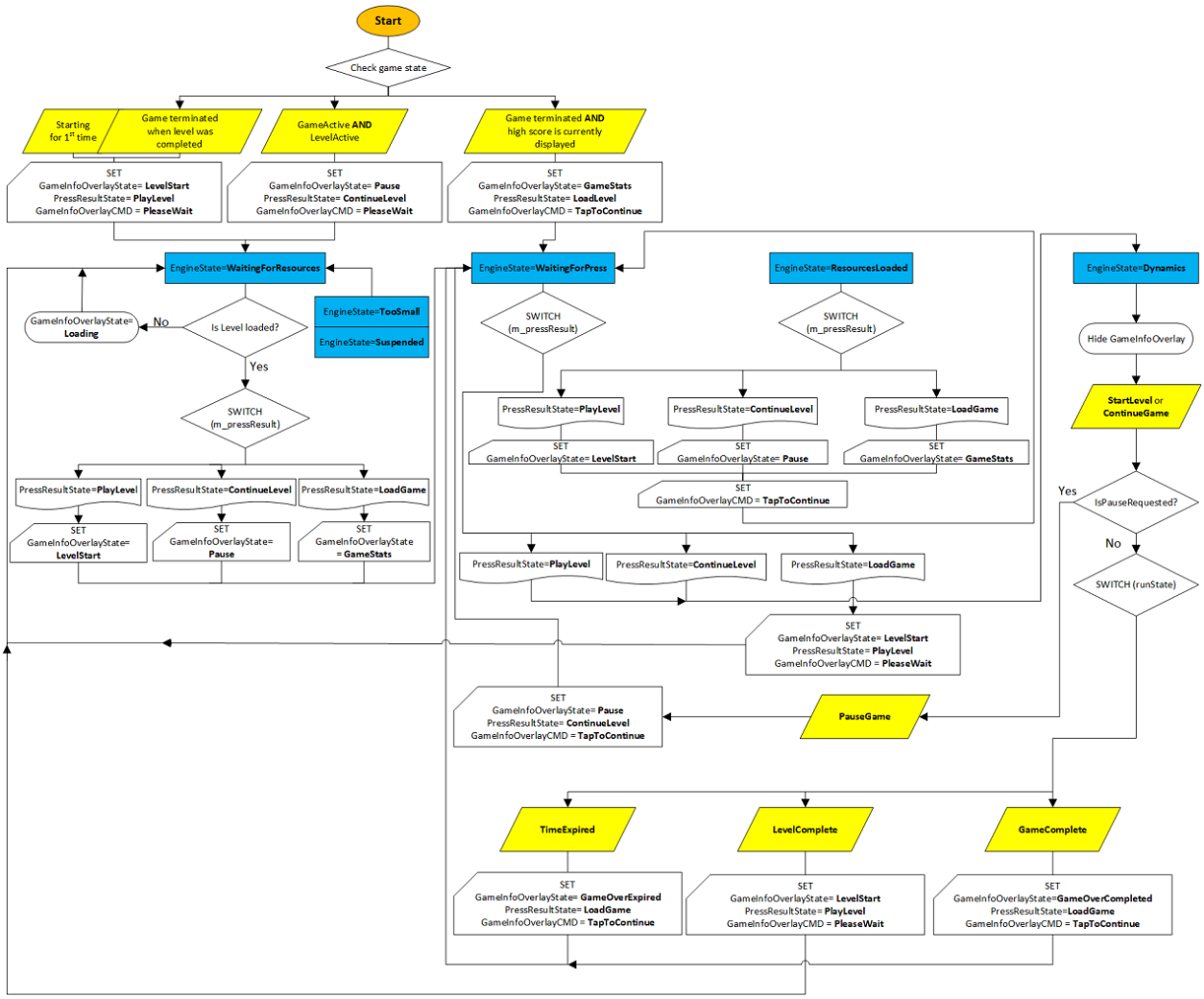
The sample game doesn't distinguish whether the game is cold starting, launching for the first time without a suspend event, or resuming from a suspended state. This is the proper design for any UWP app.

In this sample, initialization of the game states occurs in **GameMain::InitializeGameState** (an outline of that method is show in the next section).

Here's a flowchart to help you visualize the flow. It covers both initialization and the update loop.

- Initialization begins at the **Start** node when you check for the current game state. For game code, see **GameMain::InitializeGameState** in the next section.

- For more info about the update loop, go to Update game engine. For game code, go to **GameMain::Update**.



The GameMain::InitializeGameState method

GameMain::InitializeGameState method is called indirectly via the **GameMain** class's constructor, which is the result of making a **GameMain** instance within **App::Load**.

C++/WinRT

```
GameMain::GameMain(std::shared_ptr<DX::DeviceResources> const&
deviceResources) : ...
{
    m_deviceResources->RegisterDeviceNotify(this);
    ...
    ConstructInBackground();
}

winrt::fire_and_forget GameMain::ConstructInBackground()
{
    ...
    m_renderer->FinalizeCreateGameDeviceResources();

    InitializeGameState();
    ...
}
```

```
void GameMain::InitializeGameState()
{
    // Set up the initial state machine for handling Game playing state.
    if (m_game->GameActive() && m_game->LevelActive())
    {
        // The last time the game terminated it was in the middle
        // of a level.
        // We are waiting for the user to continue the game.
        ...
    }
    else if (!m_game->GameActive() && (m_game->HighScore().totalHits > 0))
    {
        // The last time the game terminated the game had been completed.
        // Show the high score.
        // We are waiting for the user to acknowledge the high score and
start a new game.
        // The level resources for the first level will be loaded later.
        ...
    }
    else
    {
        // This is either the first time the game has run or
        // the last time the game terminated the level was completed.
        // We are waiting for the user to begin the next level.
        ...
    }
    m_uiControl->ShowGameInfoOverlay();
}
```

Update game engine

The **App::Run** method calls **GameMain::Run**. Within **GameMain::Run** is a basic state machine for handling all of the major actions that a user can take. The highest level of this state machine deals with loading a game, playing a specific level, or continuing a level after the game has been paused (by the system or by the user).

In the sample game, there are 3 major states (represented by the **UpdateEngineState** enum) that the game can be in.

1. **UpdateEngineState::WaitingForResources**. The game loop is cycling, unable to transition until resources (specifically graphics resources) are available. When the async resource-loading tasks complete, we update the state to **UpdateEngineState::ResourcesLoaded**. This usually happens between levels when the level is loading new resources from disk, from a game server, or from a cloud backend. In the sample game, we simulate this behavior, because the sample doesn't need any additional per-level resources at that time.

2. **UpdateEngineState::WaitingForPress**. The game loop is cycling, waiting for specific user input. This input is a player action to load a game, to start a level, or to continue a level. The sample code refers to these sub-states via the **PressResultState** enumeration.
3. **UpdateEngineState::Dynamics**. The game loop is running with the user playing. While the user is playing, the game checks for 3 conditions that it can transition on:

- **GameState::TimeExpired**. Expiration of the time limit for a level.
- **GameState::LevelComplete**. Completion of a level by the player.
- **GameState::GameComplete**. Completion of all levels by the player.

A game is simply a state machine containing multiple smaller state machines. Each specific state must be defined by very specific criteria. Transitions from one state to another must be based on discrete user input, or system actions (such as graphics resource loading).

While planning for your game, consider drawing out the entire game flow to ensure that you've addressed all possible actions the user or the system can take. A game can be very complicated, so a state machine is a powerful tool to help you visualize this complexity, and make it more manageable.

Let's take a look at the code for the update loop.

The GameMain::Update method

This is the structure of the state machine used to update the game engine.

C++/WinRT

```
void GameMain::Update()
{
    // The controller object has its own update loop.
    m_controller->Update();

    switch (m_updateState)
    {
    case UpdateEngineState::WaitingForResources:
        ...
        break;

    case UpdateEngineState::ResourcesLoaded:
        ...
        break;

    case UpdateEngineState::WaitingForPress:
        if (m_controller->IsPressComplete())
```

```
        {
            ...
        }
        break;

    case UpdateEngineState::Dynamics:
        if (m_controller->IsPauseRequested())
        {
            ...
        }
        else
        {
            // When the player is playing, work is done by
Simple3DGame::RunGame.
            GameState runState = m_game->RunGame();
            switch (runState)
            {
            case GameState::TimeExpired:
                ...
                break;

            case GameState::LevelComplete:
                ...
                break;

            case GameState::GameComplete:
                ...
                break;
            }
        }

        if (m_updateState == UpdateEngineState::WaitingForPress)
        {
            // Transitioning state, so enable waiting for the press event.
            m_controller->WaitForPress(
                m_renderer->GameInfoOverlayUpperLeft(),
                m_renderer->GameInfoOverlayLowerRight());
        }
        if (m_updateState == UpdateEngineState::WaitingForResources)
        {
            // Transitioning state, so shut down the input controller
            // until resources are loaded.
            m_controller->Active(false);
        }
        break;
    }
}
```

Update the user interface

We need to keep the player apprised of the state of the system, and allow the game state to change depending on the player's actions and the rules that define the game.

Many games, including this sample game, commonly use user interface (UI) elements to present this info to the player. The UI contains representations of game state and other play-specific info such as score, ammo, or the number of chances remaining. UI is also called the overlay because it is rendered separately from the main graphics pipeline, and placed on top of the 3D projection.

Some UI info is also presented as a heads-up display (HUD) to allow the user to see that info without taking their eyes entirely off of the main gameplay area. In the sample game, we create this overlay by using the Direct2D APIs. Alternatively, we could create this overlay using XAML, which we discuss in Extending the sample game.

There are two components to the user interface.

- The HUD that contains the score and info about the current state of gameplay.
- The pause bitmap, which is a black rectangle with text overlaid during the paused/suspended state of the game. This is the game overlay. We discuss it further in Adding a user interface.

Unsurprisingly, the overlay has a state machine too. The overlay can display a level start or a game-over message. It's essentially a canvas on which we can output any info about game state that we want to display to the player while the game is paused or suspended.

The overlay rendered can be one of these six screens, depending on the state of the game.

1. Resource-loading progress screen at the start of the game.
2. Gameplay statistics screen.
3. Level start message screen.
4. Game-over screen when all of the levels are completed without the time running out.
5. Game-over screen when time runs out.
6. Pause menu screen.

Separating your user interface from your game's graphics pipeline allows you to work on it independently of the game's graphics rendering engine, and decreases the complexity of your game's code significantly.

Here's how the sample game structures the overlay's state machine.

C++/WinRT

```
void GameMain::SetGameInfoOverlay(GameInfoOverlayState state)
{
    m_gameInfoOverlayState = state;
```

```
    switch (state)
    {
    case GameInfoOverlayState::Loading:
        m_uiControl->SetGameLoading(m_loadingCount);
        break;

    case GameInfoOverlayState::GameStats:
        ...
        break;

    case GameInfoOverlayState::LevelStart:
        ...
        break;

    case GameInfoOverlayState::GameOverCompleted:
        ...
        break;

    case GameInfoOverlayState::GameOverExpired:
        ...
        break;

    case GameInfoOverlayState::Pause:
        ...
        break;
    }
}
```

Event-handling

As we saw in the Define the game's UWP app framework topic, many of the view-provider methods of the **App** class register event handlers. These methods need to correctly handle these important events before we add game mechanics or start graphics development.

The proper handling of the events in question is fundamental to the UWP app experience. Because a UWP app can at any time be activated, deactivated, resized, snapped, unsnapped, suspended, or resumed, the game must register for these events as soon as it can, and handle them in a way that keeps the experience smooth and predictable for the player.

These are the event handlers used in this sample, and the events they handle.

Event handler	Description
OnActivated	Handles **CoreApplicationView::Activated**. The game app has been brought to the foreground, so the main window is activated.

Event handler	Description
OnDpiChanged	Handles Graphics::Display::DisplayInformation::DpiChanged. The DPI of the display has changed and the game adjusts its resources accordingly. **Note CoreWindow** coordinates are in device-independent pixels (DIPs) for **Direct2D**. As a result, you must notify Direct2D of the change in DPI to display any 2D assets or primitives correctly.
OnOrientationChanged	Handles Graphics::Display::DisplayInformation::OrientationChanged. The orientation of the display changes and rendering needs to be updated.
OnDisplayContentsInvalidated	Handles Graphics::Display::DisplayInformation::DisplayContentsInvalidated. The display requires redrawing and your game needs to be rendered again.
OnResuming	Handles CoreApplication::Resuming. The game app restores the game from a suspended state.
OnSuspending	Handles CoreApplication::Suspending. The game app saves its state to disk. It has 5 seconds to save state to storage.
OnVisibilityChanged	Handles CoreWindow::VisibilityChanged. The game app has changed visibility, and has either become visible or been made invisible by another app becoming visible.
OnWindowActivationChanged	Handles CoreWindow::Activated. The game app's main window has been deactivated or activated, so it must remove focus and pause the game, or regain focus. In both cases, the overlay indicates that the game is paused.
OnWindowClosed	Handles CoreWindow::Closed. The game app closes the main window and suspends the game.
OnWindowSizeChanged	Handles CoreWindow::SizeChanged. The game app reallocates the graphics resources and overlay to accommodate the size change, and then updates the render target.

Next steps

In this topic, we've seen how the overall game flow is managed using game states, and that a game is made up of multiple different state machines. We've also seen how to

update the UI, and manage key app event handlers. Now we're ready to dive into the rendering loop, the game, and its mechanics.

You can go through the remaining topics that document this game in any order.

- Define the main game object
- Rendering framework I: Intro to rendering
- Rendering framework II: Game rendering
- Add a user interface
- Add controls
- Add sound

Define the main game object

Article • 10/20/2022

> **Note**
>
> This topic is part of the **Create a simple Universal Windows Platform (UWP) game with DirectX** tutorial series. The topic at that link sets the context for the series.

Once you've laid out the basic framework of the sample game, and implemented a state machine that handles the high-level user and system behaviors, you'll want to examine the rules and mechanics that turn the sample game into a game. Let's look at the details of the sample game's main object, and how to translate game rules into interactions with the game world.

Objectives

- Learn how to apply basic development techniques to implement game rules and mechanics for a UWP DirectX game.

Main game object

In the **Simple3DGameDX** sample game, **Simple3DGame** is the main game object class. An instance of **Simple3DGame** is constructed, indirectly, via the **App::Load** method.

Here are some of the features of the **Simple3DGame** class.

- Contains implementation of the gameplay logic.
- Contains methods that communicate these details.
  - Changes in the game state to the state machine defined in the app framework.
  - Changes in the game state from the app to the game object itself.
  - Details for updating the game's UI (overlay and heads-up display), animations, and physics (the dynamics).

> **Note**
>
> Updating of graphics is handled by the **GameRenderer** class, which contains methods to obtain and use graphics device resources used by the game. For more info, see **Rendering framework I: Intro to rendering**.

- Serves as a container for the data that defines a game session, level, or lifetime, depending on how you define your game at a high level. In this case, the game state data is for the lifetime of the game, and is initialized one time when a user launches the game.

To view the methods and data defined by this class, see The Simple3DGame class below.

Initialize and start the game

When a player starts the game, the game object must initialize its state, create and add the overlay, set the variables that track the player's performance, and instantiate the objects that it will use to build the levels. In this sample, this is done when the **GameMain** instance is created in **App::Load**.

The game object, of type **Simple3DGame**, is created in the **GameMain::GameMain** constructor. It's then initialized using the **Simple3DGame::Initialize** method during the **GameMain::ConstructInBackground** fire-and-forget coroutine, which is called from **GameMain::GameMain**.

The Simple3DGame::Initialize method

The sample game sets up these components in the game object.

- A new audio playback object is created.
- Arrays for the game's graphic primitives are created, including arrays for the level primitives, ammo, and obstacles.
- A location for saving game state data is created, named *Game*, and placed in the app data settings storage location specified by **ApplicationData::Current**.
- A game timer and the initial in-game overlay bitmap are created.
- A new camera is created with a specific set of view and projection parameters.
- The input device (the controller) is set to the same starting pitch and yaw as the camera, so the player has a 1-to-1 correspondence between the starting control position and the camera position.
- The player object is created and set to active. We use a sphere object to detect the player's proximity to walls and obstacles and to keep the camera from getting placed in a position that might break immersion.
- The game world primitive is created.
- The cylinder obstacles are created.
- The targets (**Face** objects) are created and numbered.
- The ammo spheres are created.
- The levels are created.

- The high score is loaded.
- Any prior saved game state is loaded.

The game now has instances of all the key components—the world, the player, the obstacles, the targets, and the ammo spheres. It also has instances of the levels, which represent configurations of all of the above components and their behaviors for each specific level. Now let's see how the game builds the levels.

Build and load game levels

Most of the heavy lifting for the level construction is done in the `Level[N].h/.cpp` files found in the **GameLevels** folder of the sample solution. Because it focuses on a very specific implementation, we won't be covering them here. The important thing is that the code for each level is run as a separate **Level[N]** object. If you'd like to extend the game, you can create a **Level[N]** object that takes an assigned number as a parameter and randomly places the obstacles and targets. Or, you can have it load level configuration data from a resource file, or even the internet.

Define the gameplay

At this point, we have all the components we need to develop the game. The levels have been constructed in memory from the primitives, and are ready for the player to start interacting with.

The best games react instantly to player input, and provide immediate feedback. This is true for any type of a game, from twitch-action, real-time first-person shooters to thoughtful, turn-based strategy games.

The Simple3DGame::RunGame method

While a game level is in progress, the game is in the **Dynamics** state.

GameMain::Update is the main update loop that updates the application state once per frame, as shown below. The update loop calls the **Simple3DGame::RunGame** method to handle the work if the game is in the **Dynamics** state.

C++/WinRT

```
// Updates the application state once per frame.
void GameMain::Update()
{
    // The controller object has its own update loop.
    m_controller->Update();
```

```
    switch (m_updateState)
    {
    ...
    case UpdateEngineState::Dynamics:
        if (m_controller->IsPauseRequested())
        {
            ...
        }
        else
        {
            // When the player is playing, work is done by
Simple3DGame::RunGame.
            GameState runState = m_game->RunGame();
            switch (runState)
            {
                ...
```

Simple3DGame::RunGame handles the set of data that defines the current state of the game play for the current iteration of the game loop.

Here's the game flow logic in **Simple3DGame::RunGame**.

- The method updates the timer that counts down the seconds until the level is completed, and tests to see whether the level's time has expired. This is one of the rules of the game—when time runs out, if not all the targets have been shot, then it's game over.
- If time has run out, then the method sets the **TimeExpired** game state, and returns to the **Update** method in the previous code.
- If time remains, then the move-look controller is polled for an update to the camera position; specifically, an update to the angle of the view normal projecting from the camera plane (where the player is looking), and the distance that angle has moved since the controller was polled last.
- The camera is updated based on the new data from the move-look controller.
- The dynamics, or the animations and behaviors of objects in the game world independent of player control, are updated. In this sample game, the **Simple3DGame::UpdateDynamics** method is called to update the motion of the ammo spheres that have been fired, the animation of the pillar obstacles and the movement of the targets. For more information, see Update the game world.
- The method checks to see whether the criteria for the successful completion of a level have been met. If so, it finalizes the score for the level, and checks to see whether this is the last level (of 6). If it's the last level, then the method returns the **GameState::GameComplete** game state; otherwise, it returns the **GameState::LevelComplete** game state.